

Reverb Algorithms

Warren L. G. Koontz

August 3, 2026

1 Introduction

Reverberation, or simply reverb, is a popular effect to apply to an audio recording. It provides the impression that the audio has been recorded in an acoustically live, but hopefully not too live, space. In such a space, sound travels from the source to the listener over multiple paths either directly or through one or more reflections. At a given moment, the listener is hearing “newer” sounds that follow a direct path from the source and “older” sounds that follow indirect paths. The oldest sounds, which have lost power but are still audible, may have originated several seconds before the newest sounds.

A reverb effect algorithm processes a sequence of audio input samples to create a corresponding sequence of audio output samples. If each input sample only affects a single output sample, there will be no reverb effect. But if input sample $x(n)$ can affect output sample $y(n)$ as well as a few seconds of future output samples, reverb is a possibility. Reverb algorithms are designed to do this.

This document describes two algorithms that process a stream of audio samples to add a reverb effect. We begin by describing a simple model of a delay line with loss, which is a component of both algorithms. We then describe the two algorithms:

- Algorithm based on the work of M. Schroeder.
- Algorithm based on a feedback delay network (FDN)

2 Delay Line With Loss

Audio signal processing algorithms often include sample delays ranging from a few samples for a simple filter to thousands of samples to simulate an echo or reverberation. In the latter cases, the sample delay is a consequence of the time required for

sound to propagate along a path between the source to the listener. For a path of length L , the time required is simply

$$T = L/c \quad (1)$$

where c is the speed of sound. If the sampling rate is f_s Hz (samples/second), then the corresponding sample delay is

$$M = \text{round}(f_s T) \quad (2)$$

The basic equation for a sample delay is simply

$$y(n) = x(n - M) \quad (3)$$

where $x(n)$ and $y(n)$ are sequences of input and output samples, respectively and M is the sample delay. If we apply the z-transform, we obtain an equally simple expression:

$$Y(z) = z^{-M} X(z) \quad (4)$$

Equation (3) and (4) model a basic delay line.

A frequency-dependant loss can be modeled as a simple low-pass filter given by

$$y(n) = dy(n - 1) + gx(n) \quad (5)$$

or, in terms of a transfer function

$$\begin{aligned} H(z) &= Y(z)/X(z) \\ &= \frac{g}{1 - dz^{-1}} \end{aligned} \quad (6)$$

The frequency response of the low-pass filter is given by

$$H(\Omega) = \frac{g}{1 - de^{-j\Omega}} \quad (7)$$

where Ω is the frequency in radians/sample. For $\Omega = 0$ (corresponding to frequency $f = 0$ Hz), the magnitude of the frequency response is

$$|H(0)| = \frac{g}{1 - d} \quad (8)$$

For $\Omega = \pi$ (corresponding to frequency $f = f_s/2$) the magnitude of the frequency response is

$$|H(\pi)| = \frac{g}{1 + d} \quad (9)$$

If the loss is due to absorption of sound by the medium, then the loss at high frequencies is considerably larger than at low frequencies. Thus we will assume that $|H(0)| = 1$, i.e., negligible loss at low frequency. Under this assumption, $g = 1 - d$ and

$$|H(\pi)| = \frac{1 - d}{1 + d} \quad (10)$$

Under these assumptions, we will refer to the parameter d as the *damping*. The high-frequency attenuation $|H(\pi)|$ might also be expressed in terms of a high-frequency absorption A_{HF} expressed in dB/meter. Then the attenuation in dB for a path of length L is simply $A_{\text{HF}}L$ and the attenuation is given by

$$|H(\pi)| = 10^{-A_{\text{HF}}L/20} \quad (11)$$

From Equations (10) and (11) we obtain the following equation for the damping as a function of the high frequency absorption and the path length:

$$d = \frac{1 - 10^{-A_{\text{HF}}L/20}}{1 + 10^{-A_{\text{HF}}L/20}} \quad (12)$$

Under all of these assumptions, the frequency response of the low-pass filter is

$$H(\Omega) = \frac{1 - d}{1 + de^{-j\Omega}} \quad (13)$$

with the damping d given by Equation (12). Figure 1 is a plot of the frequency response for $L = 20$ meters and $A_{\text{HF}} = 0.3$ dB/meter. Figure 2 is a plot of the impulse response.

Putting the low-pass filter together with the simple delay (as shown in Figure 3), we get the following model for a delay line with frequency-dependant loss:

$$y(n) = dy(n - 1) + (1 - d)x(n - M) \quad (14)$$

or

$$H(z) = \frac{(1 - d)z^{-M}}{1 - dz^{-1}} \quad (15)$$

Note that the parameters d and M are both determined from the high-frequency absorption A_{HF} , the path length L , the speed of sound c , and the sampling frequency f_s . The frequency response of the delay line is the same as that of the low-pass filter and the impulse response differs only by a time delay.

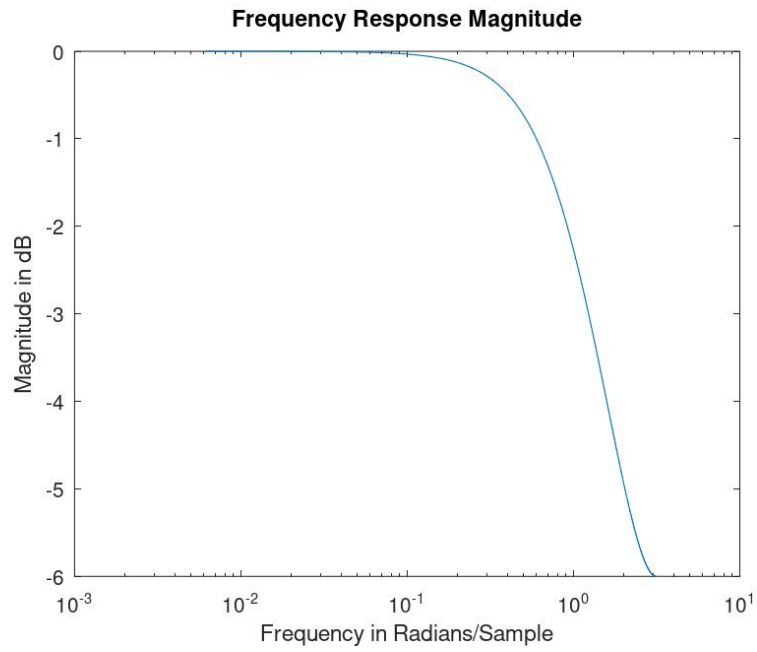


Figure 1: Frequency Response of Low-Pass Filter

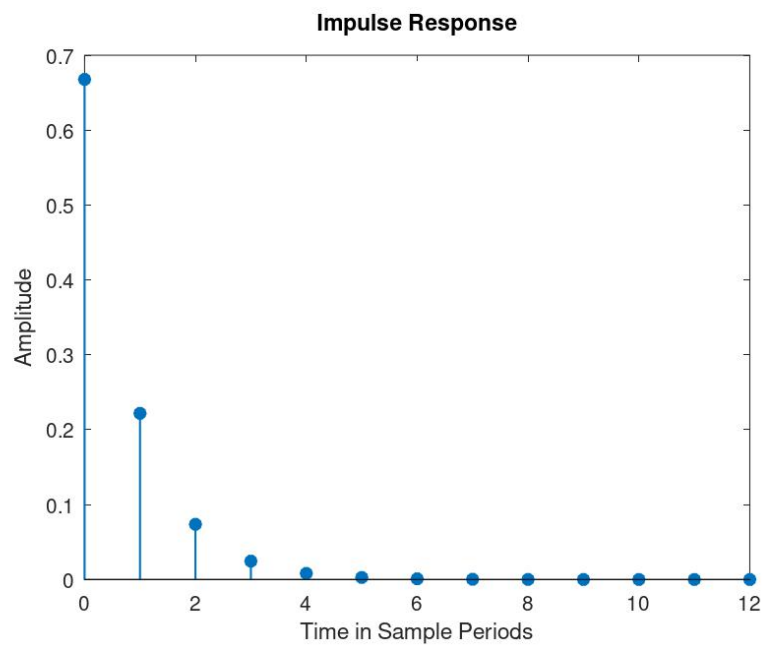


Figure 2: Impulse Response of Low-Pass Filter

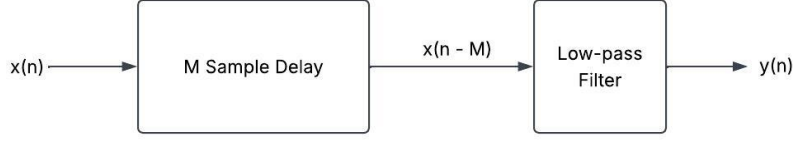


Figure 3: Delay Line With Frequency-Dependent Loss

3 Schroeder Reverb

Schroeder’s algorithm uses two basic components: comb filters and all-pass filters. This document describes each of these components, including some of my own variations, and how they are combined in the final algorithm.

3.1 The Feedback Comb Filter

The transfer function and difference equation for a basic feedback comb filter are:

$$h(z) = \frac{1}{1 - Rz^{-M}} \quad (16)$$

$$y(n) = x(n) + Ry(n - M) \quad (17)$$

The difference equation states that the output is the sum of the current input plus a previous output, modified by a “reflection” parameter R . The sample delay M accounts for a propagation delay for the previous output sample. Figure 4 shows the impulse response of the basic feedback comb filter - a series of echos. For this and succeeding examples the reflection is set to $R = 0.5$ and the sample delay is based on the length of a small area room.

The low-pass feedback comb filter is derived from the basic form by replacing the sample delay with a lossy delay line with damping d and sample delay M as shown in Figure 5. The output samples are related to the input samples by a pair of equations:

$$\begin{aligned} s(n) &= ds(n - 1) + (1 - d)y(n - M) \\ y(n) &= x(n) + Rs(n) \end{aligned} \quad (18)$$

where $s(n)$ is the output of the lossy delay line. The impulse response of the low-pass version is similar to that of the basic version, except for a small broadening of the echos as shown in a close-up in Figure 6.

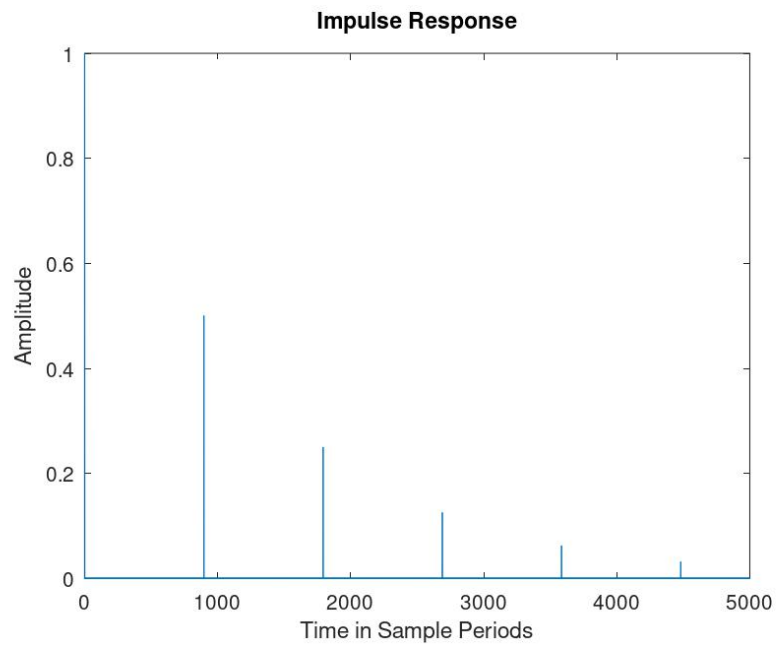


Figure 4: Impulse Response of Feedback Comb Filter

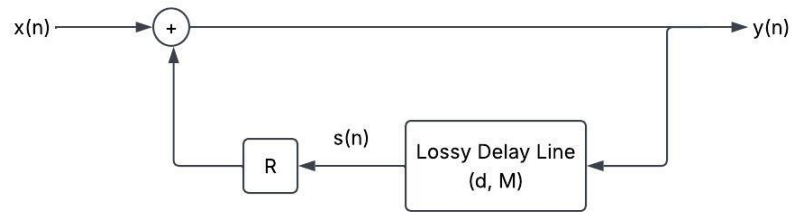


Figure 5: Low-Pass Feedback Comb Filter

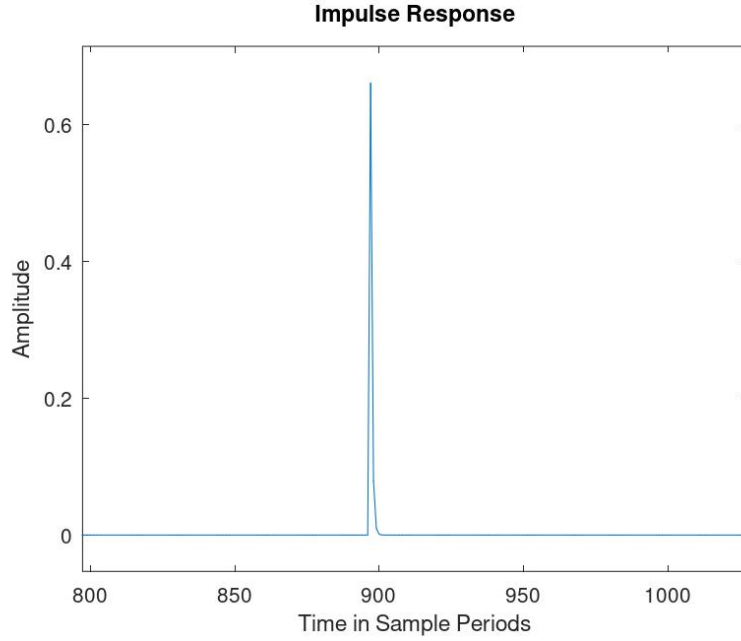


Figure 6: Detail of Impulse Response of Low-Pass Feedback Comb Filter

3.2 All-Pass Filters

The all-pass filter also comes in a basic and low-pass version, although “low-pass all-pass” is a bit contradictory. The transfer function and difference equation for the basic all-pass filter are

$$h(z) = \frac{-R + z^{-M}}{1 - Rz^{-M}} \quad (19)$$

$$y(n) = -Rx(n) + x(n - M) + Ry(n - M) \quad (20)$$

The magnitude of the frequency response is 1 for all frequencies (hence the name) and the impulse response is a series of echos similar to those of the comb filters.

The low-pass version of the filter is obtained by replacing the sample delay with a lossy delay line as shown in Figure 7. At this point, the associated equations follow easily from the figure.

$$\begin{aligned} s(n) &= ds(n - 1) + (1 - d)[x(n - M) + Ry(n - M)] \\ y(n) &= -Rx(n) + s(n) \end{aligned} \quad (21)$$

Figure 8 shows a typical impulse response of the filter (the initial negative response is

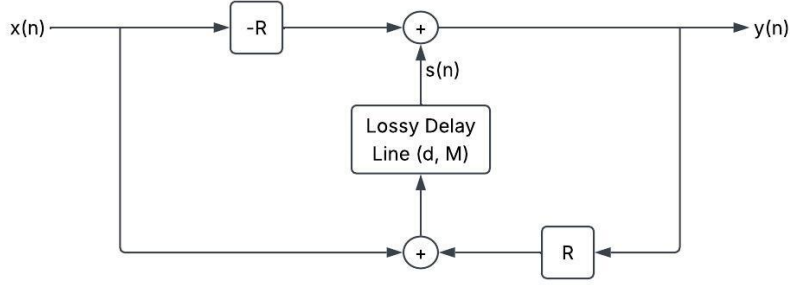


Figure 7: Low-Pass All-Pass Filter

masked by the vertical axis). Again, the echos are slightly broadened by the low-pass processing.

3.3 The Algorithm

My version of Schroeder’s reverb consists of a bank of low-pass comb filters followed by a cascade of low-pass all-pass filters as shown in Figure 9. In the version I have coded, there are 8 comb filters and 4 all-pass filters. Each input sample is fed to all of the comb filters. The outputs of the comb filters are summed and the sum is fed to the first all-pass filter in the cascade. The output of each all-pass filter is fed to the next and the output of the last is the output sample.

4 Feedback Delay Network

The FDN can be described as an extension of the standard state space model of a linear time-invariant system (LTIS). For discrete time systems with a single input stream and a single output stream, the state space model is

$$\begin{aligned}\mathbf{x}(n+1) &= \mathbf{A}\mathbf{x}(n) + \mathbf{B}u(n) \\ y(n) &= \mathbf{C}\mathbf{x}(n) + Du(n)\end{aligned}\tag{22}$$

In this notation $u(n)$ and $y(n)$ are the input and output streams, respectively and the column vector $\mathbf{x}(n)$ is the *state vector*. If the dimension of $\mathbf{x}$ is N , then $\mathbf{A}$ is a N by N state transition matrix, $\mathbf{B}$ is a N by 1 distribution vector, and $\mathbf{C}$ is a 1 by

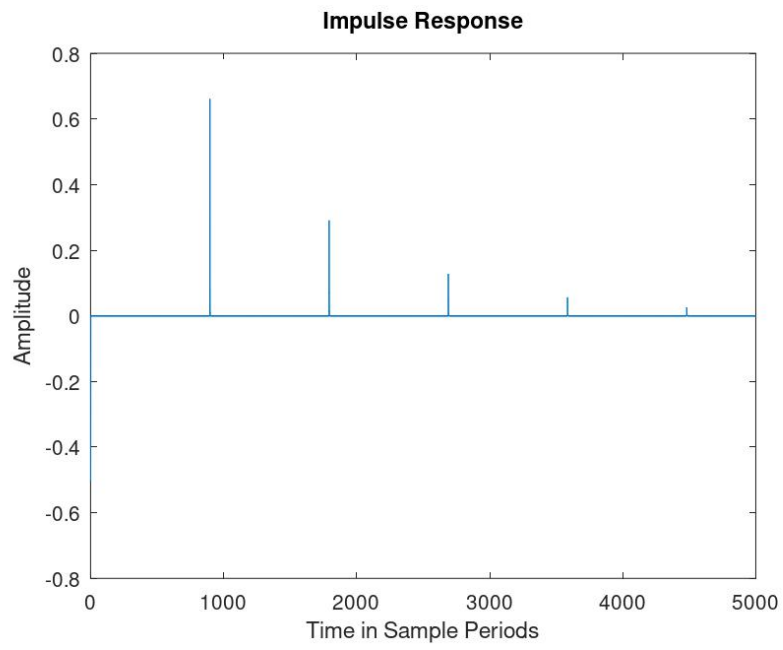


Figure 8: Low-Pass All-Pass Filter Impulse Response

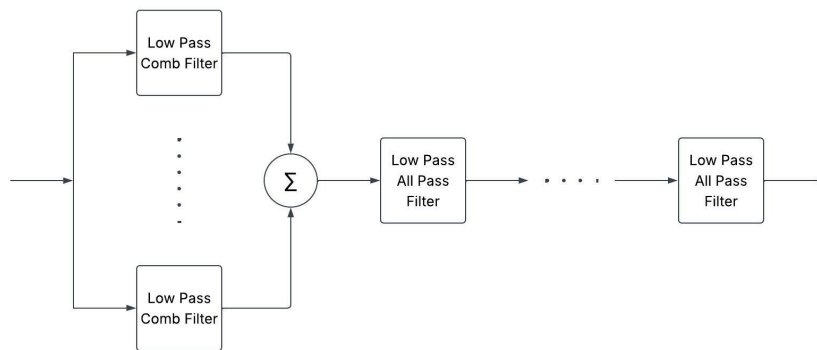


Figure 9: Schroeder Reverb Structure

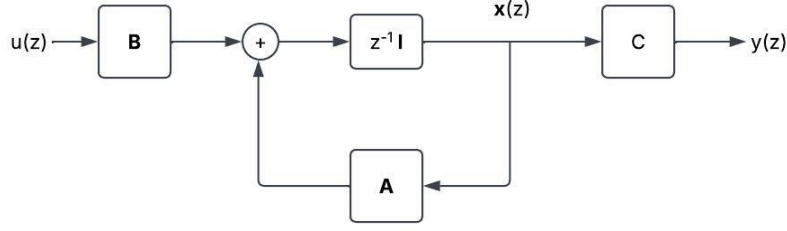


Figure 10: Basic State Space Model

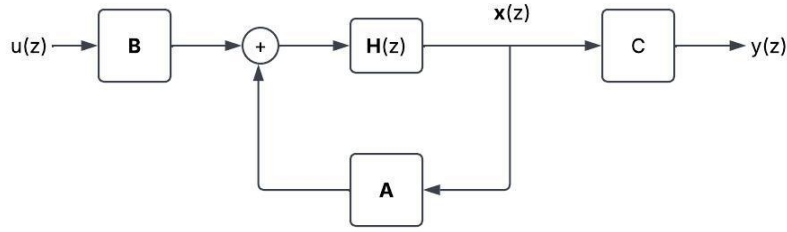


Figure 11: Enhanced State Space Model

N accumulation vector. In the z domain, Equation (22) becomes

$$\begin{aligned} z\mathbf{x}(z) &= \mathbf{A}\mathbf{x}(z) + \mathbf{B}u(z) \\ y(z) &= \mathbf{C}\mathbf{x}(z) + Du(z) \end{aligned} \quad (23)$$

Figure 10 illustrates the state space model as a feedback system (assuming $D = 0$).

Figure 11 shows an expanded version of the state space model where the one sample delay ($z^{-1}\mathbf{I}$) is replaced by a more general matrix $\mathbf{H}(z)$. To obtain a FDN, let the various matrices be defined as follows:

$$\mathbf{A} = \frac{R}{N}\mathbf{J} - \mathbf{I} \quad (24)$$

$$\mathbf{B} = [1 \ 1 \ \dots \ 1]^T \quad (25)$$

$$\mathbf{C} = \frac{1}{N} [1 \ 1 \ \dots \ 1] \quad (26)$$

where $\mathbf{J}$ is a $N \times N$ matrix of ones and R is a “reflection” parameter. $\mathbf{H}(z)$ is a

$N \times N$ diagonal matrix with diagonal elements given by

$$H_{kk}(z) = \frac{(1 - d_k)z^{-M_k}}{1 - d_k z^{-1}} \quad (27)$$

which models a bank of lossy delay lines.

4.1 Frequency Domain Analysis of FDN

The transfer function for the FDN is

$$\begin{aligned} h(z) &= \frac{y(z)}{u(z)} \\ &= \mathbf{C}[\mathbf{I} - \mathbf{H}(z)\mathbf{A}]^{-1}\mathbf{H}(z)\mathbf{B} \end{aligned} \quad (28)$$

The FDN frequency response $h(\Omega)$ is obtained from Equation (28) with $z = e^{-j\Omega}$. Figure 12 shows the magnitude of the frequency response for a reasonable example of a FDN. The complexity of the frequency response reflects the fact that the FDN is a very high-order filter. In fact, the frequency response does not seem particularly useful except for the fact that it can be transformed into the impulse response using the inverse FFT as shown in Figure 13.

4.2 Time Domain Analysis of FDN

For implementation purposes it is more useful to return to the time domain. The following results come from the definitions of $\mathbf{A}$, $\mathbf{B}$, and $\mathbf{C}$. First of all, the output $y(n)$ is simply the average of the components of the state vector $\mathbf{x}$.

$$y(n) = \frac{1}{N} \sum_{k=0}^{N-1} x_k(n) \quad (29)$$

The inputs to the lossy delay lines, $v_k(n)$ are given by

$$\begin{aligned} v_k(n) &= u(n) + \frac{R}{N} \sum_{l=0}^{N-1} x_l(n) - x_k(n) \\ &= u(n) + Ry(n) - x_k(n) \end{aligned} \quad (30)$$

Figure 14 shows what the FDN looks like based on the chosen definitions. From the figure and previous discussion, it is easy to see that the $x_k(n)$ satisfy

$$x_k(n) = d_k x_k(n-1) + (1 - d_k) v_k(n - M_k) \quad (31)$$

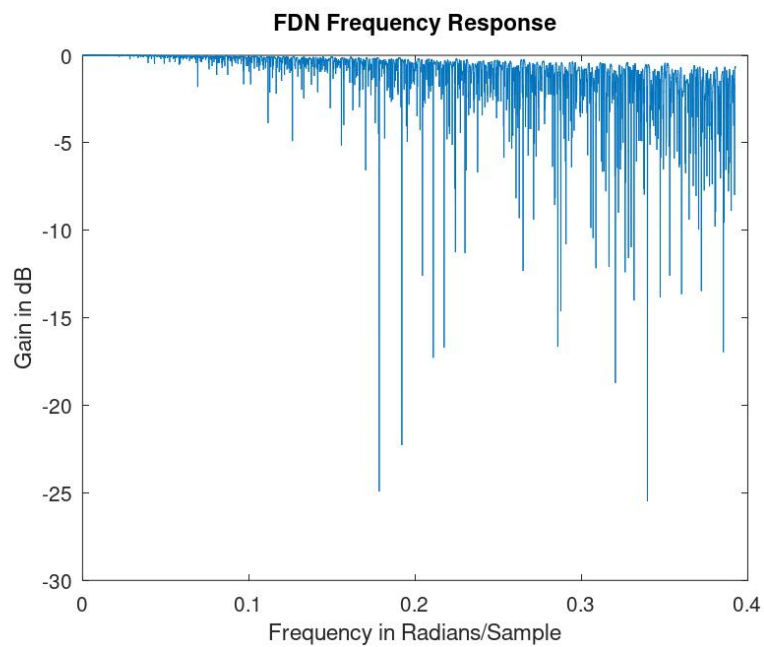


Figure 12: FDN Frequency Response

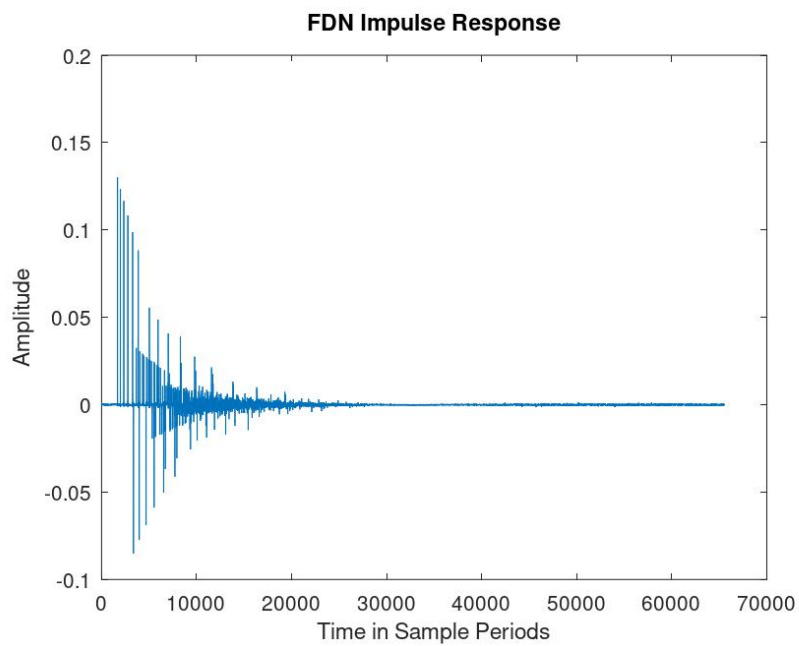


Figure 13: FDN Impulse Response

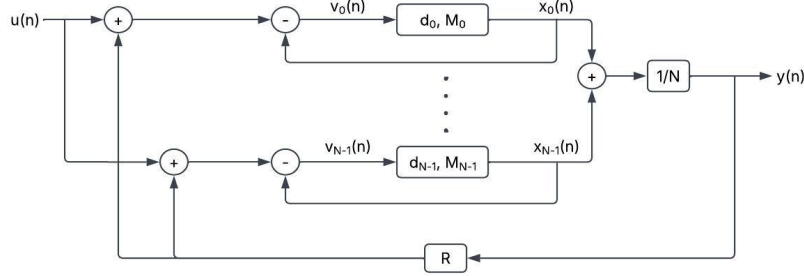


Figure 14: Scalar Model of a Feedback Delay Network

5 Implementation Considerations

I have implemented both the Schroeder and FDN reverb algorithms as C++ classes. The Schroeder reverb includes a LPC filter class and a LPAP filter class. Each class has a method called “step” that takes an input audio sample and returns an output audio sample. For example, the LPC filter is governed by Equation (18), which is repeated here:

$$\begin{aligned} s(n) &= ds(n-1) + (1-d)y(n-M) \\ y(n) &= x(n) + Rs(n) \end{aligned} \quad (32)$$

Here is the C++ code for the LPC filter step method:

```

1 float LPcombfilter::step(float insample)
2 {
3     // Update state variable
4     unsigned oldest = (m_current + m_sampledelay) % m_buffer.
        size();
5     m_state = m_damping * m_state + (1.0 - m_damping) *
        m_buffer[oldest];
6
7     // Calculate output
8     float outsample = insample + m_reflection * m_state;
9
10    // Update delay buffer
11    m_buffer[m_current] = outsample;
12    m_current = m_current == 0 ? m_buffer.size() - 1 :
        m_current - 1;
13

```

```

14     \\ Return output
15     return outsample;
16 }

```

Lines 5 and 8 are the C++ version of Equation (18). The rest of the code has to do with a circular buffer that contains previous output samples. The sample delay variable corresponds to the variable M in Equation (18). The LPAP filter step method is derived from Equation (21) in a similar manner. The step method for the Schroeder reverb combines the LPC and LPAP step methods.

The FDN step method is a bit more complicated, but still follows from the FDN equations straightforwardly. Here are the equations again:

$$\begin{aligned}
 x_k(n) &= d_k x_k(n-1) + (1 - d_k) v_k(n - M_k) \\
 y(n) &= \frac{1}{N} \sum_{k=0}^{N-1} x_k(n) \\
 v_k(n) &= u(n) + Ry(n) - x_k(n)
 \end{aligned} \tag{33}$$

and the corresponding code:

```

1  // Update state variables
2  for (unsigned k = 0; k < ORDER; k++)
3  {
4      unsigned old = (m_current + m_sampleDelay[k]) %
5          m_delayLine[k].size();
6      x[k] = m_damping[k] * x[k] + (1.0 - m_damping[k]) *
7          m_delayLine[k][old];
8  }
9
10 // Calculate output
11 double audioOut = 0.0;
12 for (unsigned k = 0; k < ORDER; k++)
13     audioOut += x[k];
14 audioOut /= double(ORDER);
15
16 // Update delay lines with new input sample
17 for (unsigned k = 0; k < ORDER; k++)
18     m_delayLine[k][m_current] = audioIn + m_reflection *
19         audioOut - x[k];
20
21 // Update index of oldest sample in delay line

```

```

19     m_current = m_current == 0 ? m_delayLine[0].size() - 1 :
        m_current - 1;
20
21     // Return output
22     return audioOut;

```

Both the Schroeder reverb and the FDN reverb use the same set of user input parameters, which are listed here:

- Room Area (sq m) - used to determine a maximum path length (m) that is used to determine the sample delays M_k .
- High Frequency Absorption (dB/m) - used to determine the damping values d_k .
- Reflection - the value of R .

The maximum path length, which determines the maximum sample delay, is derived from the room area based on the following, rather flakey reasoning. First assume that the room is a golden rectangle with width W and length ΦW where

$$\Phi = \frac{1 + \sqrt{5}}{2} \quad (34)$$

Since the area of the room is ΦW^2 , the length of the room can be determined from the room area:

$$\begin{aligned}
 L &= \Phi W \\
 &= \Phi \sqrt{A_R / \Phi} \\
 &= \sqrt{\Phi A_R}
 \end{aligned} \quad (35)$$

The sample delays M_k are based on a sequence of path segments ranging in length from L down to $L/10$ exponentially.

$$L_k = \alpha^k L \quad (36)$$

where

$$\alpha^{N-1} = 0.1 \quad (37)$$

The sample delays are given by

$$M_k = \text{round}(f_s L_k / c) \quad (38)$$

where f_s is the sampling rate and c is the speed of sound. The path segment lengths L_k can also be used to compute the damping values.

$$d_k = \frac{1 - 10^{-A_{HF}L_k/20}}{1 + 10^{-A_{HF}L_k/20}} \quad (39)$$

where A_{HF} is the high frequency absorption coefficient.

The C++ classes all contain methods to update the sample delays and damping values when the user varies one or more of the parameters. I have used these classes together with the JUCE Framework to create VST3 reverb plug-ins. These plug-ins also allow the user to vary the wet/dry mix and to adjust the overall gain.